

Project Title: FareWise

Project Summary:

FareWise is a flight booking time web application that can help users understand historical air travel patterns for specific routes. Users can be able to select origin airports, destination airports, and travel preferences such as weekdays, weekends, and the holiday season. The application will be able to access historical fare data and display how ticket prices change based on how many days before the departure date. This will allow people to see how air travel typically behaves, allowing users to plan their trip on their own terms while saving money along the way. Fairwise will present this information through route-specific price trends based on the user input and will take into consideration booking windows and historical fee comparisons.

There will also be a comparison feature to help compare different time ranges with each other, seeing which best suits their travel preferences. Users will be able to support filtering through factors such as airline, departure day, and type of flight(nonstop, connecting, or international). By combining route information with historical pricing data, this application will give users a more detailed view of their plans and show how prices will vary across different conditions, making it easier to have a smoother booking experience for a trip.

Creative Feature:

A creative component of FareWise will be an interactive booking window tool that processes historical aviation data and identifies specific routes where the lowest prices are available. The results will be presented through an interactive visualization that allows users to change key features like airlines and duration of flight, and the chart will adjust accordingly. This component steps away from basic chart display, as this component will require data transformation, aggregation, filtering, and dynamic visualization rather than simply displaying values from the database.

Usefulness:

Farewise is useful because it helps people planning trips to make more informed decisions about when to purchase airline tickets. Ticket prices are a major factor when planning vacations; this can be a way to take the stress off the price of airline tickets to help plan a better experience while actually vacationing. Most travelers can see the current price of a flight, but may not know whether the price is relative to other tickets. Fairwise will address this inconvenience by displaying specific buying patterns around the time they are looking for and how they have changed throughout the course of the month.

Similar applications do exist, such as KAYAK, where they provide a suggestion on whether the price is good relative to other tickets but never show which periods exactly can be seen at the lowest price; only when browsing for current dates do they suggest how well they match up to other tickets. Fairwise differs by placing more emphasis on transparency and giving suggestions before users have to go and search manually for themselves. This allows users to see how prices changed as the departure date approached and understand why a certain booking window may be more favorable for a specific route.

Realness:

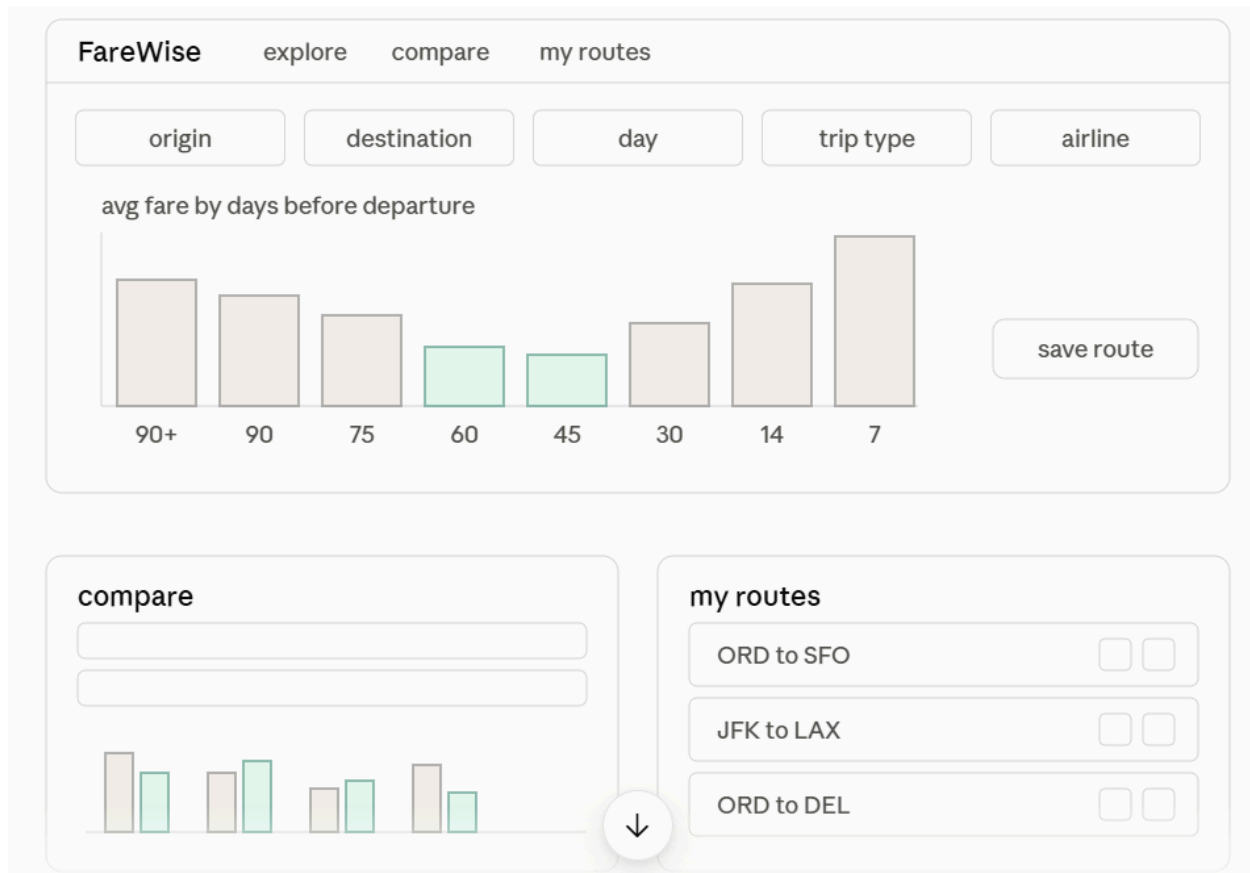
[Kaggle Dataset](#)

[Bureau of Transportation Statistics Dataset](#)

Our first source comes as a dataset from Kaggle with historical flight pricing data. This dataset contains historical flight information along with factors such as airline, source airport, destination airport, departure date, arrival time, duration, number of stops, and ticket price. This will allow us to analyze how ticket prices vary depending on routes, airlines, travel dates, and other characteristics. This dataset is in CSV format and also contains more than 10,000 flight records with many attributes that we can analyze. Each observation contains information about a flight search, including the search date, flight date, origin, destination, fare information, travel duration, remaining seats, travel distance, and more. With FareWise, we will group this information based on how many days before departure the fare was observed and calculate averages accordingly.

Our second source is the Bureau of Transportation Statistics, which is a government-provided dataset that has historical airline ticket information. This data is available in structured formats through CSV files and has more than 1,000 records. It also captures information such as origin and destination airports, airlines, ticket fares, number of passengers, and other travel information. We will use this dataset to provide additional historical information about routes, airlines, fares, and travel patterns. We hope to transform these two datasets and make connections through common attributes between them.

Functionality



Data that users add:

- A user creates an account by filling out a form with a username, email, password, and home airport when they first sign up.
- A user saves a route by filling out a form with an origin airport, destination airport, trip type, target price, and an optional note when they want to keep track of a trip.
- The app adds a recommendation row (route, best booking window, average fare, number of observations) to a cache table when a user saves a route or refreshes it. A stored procedure does this.
- The app adds a row to an archive table when a saved route is deleted. A trigger does this.

Data that users update:

- A user edits a saved route to change the target price, the note, or the filters when their plans change.
- A user edits their profile to change their home airport or password.

Data that users delete:

- A user deletes a saved route when they stop tracking that trip.

- A user deletes their account, which also removes their saved routes.

Data that users retrieve:

- A user picks an origin, destination, and filters, and sees the average fare grouped by how many days before departure the fare was recorded.
- A user enters a second set of filters and sees both results on the same chart.
- A user opens their saved routes page and sees each route with its cached recommendation.
- A user types into an airport field and gets matching airports from the airports table.

The fare data, airports, and airlines are loaded once and are never changed by users. All of the add, update, and delete operations happen on tables the user owns.

UI mockup

The mockup has three screens. The first is drawn out, and the other two are labeled boxes since they reuse the same layout.

Route explorer: a row of filter fields at the top for origin, destination, departure day, trip type, and airline. Under that is a bar chart. The x axis is days before departure (90+, 90, 75, 60, 45, 30, 14, 7) and the y axis is the average fare. The cheapest bars are shaded to show the best booking window. There is a Save route button in the corner.

Compare: the same screen with two rows of filters and two sets of bars on the chart.

My routes: a list of the user's saved routes, each with an edit and a delete button.

Work distribution

Anish, data and schema: cleans both datasets, writes the table definitions, normalizes the schema, loads the database, makes the ER diagram, and runs the index analysis with query costs before and after each index. He owns all of the table definitions.

Eshaan, queries: writes both advanced queries, the logic that groups fares into booking windows, and the join between the Kaggle fare data and the Bureau of Transportation Statistics data. He also writes the endpoints that return the chart data and tunes those queries with Anish's indexes.

Krishay, frontend: builds the pages, the filter bar, the chart, the compare view, and the saved routes page, plus the code that turns query results into chart data.

Karthik, writes and deployment: builds accounts and login, the saved routes table, the add, edit, and delete endpoints, the stored procedure that finds the best booking window, the trigger that archives deleted routes, and the transaction around the procedure. Also handles deployment and keeping everyone's setup consistent.

Anish and Eshaan will agree on table and column names in the first week so the schema and the queries stay in sync.